

Version: promotional-standardization draft

GO-1 to GO-5

*Proposal for standardization, popularization and technical evaluation of local LLM/LMM
benchmark tasks*

*Proposal for standardization, popularization and technical evaluation of local LLM/LMM
benchmark tasks*

Dr. Jovan Ivković¹

Methodological essay / scientific paper

1. Email: jovan.ivkovic@its.edu.rs, ORCID: <https://orcid.org/0000-0001-5236-9529>

This document formalizes the GO-1 to GO-5 benchmark set as a small but methodologically dense framework for evaluating local and edge LLM/LMM systems under conditions of strict formal accuracy, multilingual instruction following, structured output, deterministic instance generation, and practical code executability.

It documents the origin of the benchmark, its design philosophy, the standardized response structure, scoring rules, typical errors, a proposed protocol for public use, and an annex with canonical task formulations from GO-1 to GO-5.

Abstract

GO-1 to GO-5 constitute a purpose-designed benchmark set for evaluating local and edge LLM/LMM systems in conditions where generic leaderboards and arenas are insufficient. These tasks were not created as demonstration problems, but as a response to the practical need to compare quantized models, running on limited hardware and under different inference settings, according to real usability: strict accuracy, format discipline, quality of intermediate steps, code quality, deterministic instance generation, and transparency of machine-checkable output. This paper defines the origin of the benchmark, its conceptual basis, the scope of each individual probe, the score quantification system, the penalty logic for typical errors, the execution protocol, and recommendations for popularization and standardized application. The paper shows that a small number of carefully constructed tasks can reveal substantive differences among models much more directly than broad but less focused benchmark collections. The GO set is designed to measure what is decisive in practice: whether a model is not only persuasive, but also verifiably accurate, structured, generatively disciplined, and technically usable.

Keywords: local LLM, edge AI, benchmark methodology, multilingual evaluation, formal accuracy, structured output, bitmask reasoning, code-grounded evaluation.

Executive summary

- The GO benchmark set is intended for situations in which the user is not interested only in an average leaderboard score, but in the real usability of a model in local work.
- Together, the five tasks cover formal logic, exact numerical reasoning, spatial-algebraic manual calculation, combinatorics and search in Zig, as well as deterministic generation of a formal instance, forensic reconstruction, and uniqueness verification in a modern Zig 0.16.x+ environment.
- The benchmark is deliberately multilingual and strictly structured. The Serbian language, precise output syntax, and mandatory A/B/C/JSON sections constitute a deliberately designed test load.
- Benchmark standardization requires not only fixed task texts, but also a clear scoring protocol, preservation of all reruns, transparent selection of the representative best result per model, and the possibility of audit variants that change the instance without changing the methodology.

1. Introduction: why this benchmark was needed

The dominant public benchmark ecosystem measures general model capabilities on large task corpora very well, but it measures relatively weakly what proves decisive in real local work: the formal accuracy of auxiliary numbers, response discipline, quality of machine-checkable output, stability under quantization, and the ability to produce code close to real executability.

In local and edge scenarios, a model is not an abstract intelligence but a working tool. It runs under a limited RAM/VRAM budget, with CPU/GPU offload, variable prompt templates, different quantizations, and often without external assistance. Under such conditions, a model that is average overall but formally undisciplined may be less useful than a more modest model that is verifiably accurate.

GO-1 to GO-5 emerged as a response to precisely that problem. The goal was not to identify the most advertised model, but to introduce a small set of tasks that would measure what the user of a technical model actually wants: whether the model understands, computes, follows the format, provides correct JSON, and produces code that is closer to a tool than to decoration.

2. Conceptual origin of the benchmark

The GO-1 to GO-5 benchmark set arose from an iterative process of local testing of open and quantized models of different sizes, architectures, and inference regimes. During that process it became obvious that standard public benchmark signals were not sufficient to explain why a model performs excellently in one table, yet disappointingly in real technical work.

The basic idea in constructing the set was threefold. First, each task must have a clearly defined correct outcome and a sufficiently small dimension for a human evaluator to verify the result manually. Second, each task must force the model to reveal more than one capability at once: logic, calculation, discipline, code, and JSON. Third, the task set must be sufficiently diverse so that a model cannot pass merely because it is strong in one type of problem.

For this reason, the GO set is neither a purely mathematical, nor a purely coding, nor a purely linguistic benchmark. It is a layered set that combines formal logic, discrete dynamics, linear algebra, systems programming, and a strict output contract. Precisely this combination makes it suitable for the evaluation of sovereign local models.

3. What the benchmark should show and what it should help with

The primary purpose of the benchmark is not academic decoration, but practical differentiation among models. The GO set, including the new GO-5, should help a researcher, engineer, or advanced user answer the following questions: which model truly performs well under quantization; which model better respects structured output; which model has real coding potential; and which model is best as a reference, as a workhorse, or as a fast filter.

The second purpose of the benchmark is the standardization of internal and external reporting. If multiple testers use the same set, the same response sections, the same scoring rules, and the same result-reporting method, then comparisons become more transparent. This opens the possibility of using the GO set both for promotional purposes and for serious technical evaluations.

The third purpose of the benchmark is educational. A well-constructed benchmark is not only a measurement tool, but also an explanation. Through the GO tasks, it becomes clear why formal details matter, why JSON is not decoration, why the language of the task changes model behavior, and why a small syntactic error in code can invalidate an otherwise elegant answer.

4. Core architectural principles of the GO set

Principle	Rationale
Small dimension, high verifiability	Each task is small enough for an evaluator to verify manually, yet dense enough to expose deep errors.
Multilayered evaluation	The final solution alone is not scored; intermediate steps, formal structure, code, and JSON are also evaluated.
Multilingual load	The Serbian language introduces an additional test load and reduces reliance on memorized English benchmark patterns.
Practical code	The required programming languages and styles are not accidental: Python, C, and Zig deliberately test different levels of formal control.
Transparent penalty logic	Errors in auxiliary numbers, JSON, or syntax must not be hidden behind an attractive narrative.
Compatibility with local work	The benchmark must be relevant to quantized models, hybrid CPU/GPU execution, and edge environments.

Table 1. Fundamental design principles of the benchmark set.

5. Why language, syntax, and response structure are part of the test

The GO set is deliberately formulated in Serbian. This is not an aesthetic addition, but a methodological decision. Models are predominantly trained, fine-tuned, and benchmarked on English-language tasks, so the use of Serbian reduces the likelihood that the model will merely recognize a previously seen pattern without genuine understanding.

Mandatory sections such as A, B, C, and RESULTS_JSON are introduced in order to separate logic, analysis, code, and machine-checkable output. This structure prevents the model from hiding an error behind a lengthy writing style. If the model made an error, the evaluator can immediately see whether the problem arose in logic, numerics, implementation, or the final JSON object.

The required programming languages were not chosen at random. Python tests exact calculation and straightforward verification. C tests understanding of the memory model, indexing, and procedural implementation. Zig tests strict syntax, types, compatibility with the current standard library model, and the ability to build a small but realistic solver or verifier. For this reason, the syntax of the response language is not a secondary issue, but a central part of the benchmark philosophy.

6. Detailed description of the benchmark tasks

6.1 GO-1 — logical correction of a permission matrix with a minimal number of changes

GO-1 is a test of formal-logical reasoning over constraints. The model receives a matrix of roles and permissions and a set of rules that interconnect permission bits. The task is not only to detect violations, but to find a minimal set of bit-flip changes that brings all constraints into a satisfied state, to present the final matrix, and to explain why the solution is minimal and unique.

This test exposes models that can recognize local violations but cannot reason globally. In practice, it often happens that a model fixes one constraint and thereby violates another, or finds a valid solution with more changes than necessary. GO-1 therefore measures not only logic, but also optimization discipline.

The required response structure is: A-VIOLATIONS, B-CORRECTION, C-PROOF_OF_MINIMALITY, and RESULTS_JSON. This arrangement forces the model to separate diagnosis from correction and correction from proof.

GO-1 scoring element	Points / 10
Correct identification of all violated constraints	2.0
Correct minimal number of bit-flip changes	2.0
Correct set of changes and correct final matrix	2.5
Proof of minimality	2.0
Claim of uniqueness and its explanation	1.0
JSON discipline	0.5

Table 2. GO-1 rubric and score distribution.

6.2 GO-2 — LIF neuron, exact arithmetic, and proof of invariant

GO-2 tests the model's ability to simultaneously keep under control the discrete dynamics of a system, a formal proof, an exact tabular representation, and verifiable Python code. The model receives the equations of a LIF neuron, an input sequence, and an initial condition. It is required to prove the invariant $0 \leq v_t < 1$, calculate all steps, and deliver the resulting JSON object.

The particular value of GO-2 lies in the fact that it penalizes models that “understand the story” but corrupt the numbers. A large number of models can provide a convincing explanation of the dynamics and make an error

only in the final decimals or in one threshold case. In technical use, precisely such errors are the most dangerous, because it is easy to overlook that the narrative is elegant while the number is wrong.

The response structure A-DERIVATION, B-TABLE, C-CODE, and RESULTS_JSON deliberately forces the model to maintain consistency between the explanation, the manual calculation, and the programmatic output.

GO-2 scoring element	Points / 10
Correct proof of the invariant	2.5
Correct fractions by step	2.5
Correct decimals by step	1.5
Correct table and spike count	1.5
Python code that reproduces the result	1.5
JSON discipline	0.5

Table 3. GO-2 rubric and score distribution.

6.3 GO-3 — valid 2D convolution, manual result, and C implementation

GO-3 combines manual calculation, linear algebra, spatial intuition, and systems programming. The model must rotate the kernel by 180 degrees, manually compute every output element, explain the output dimensions and the number of operations, and then write a C implementation of valid 2D convolution.

This test is highly effective in exposing models that know the concept of convolution only superficially. Typical errors include confusing convolution and correlation, incorrectly rotating the kernel, inaccurate summation of intermediate products, a poor row-major explanation, or C code that looks professional but is in fact unreliable.

GO-3 is particularly valuable because it combines the manual and the coding layers. The model cannot pass on the basis of only one of these two layers.

GO-3 scoring element	Points / 10
Correct manual output	3.0
Correct intermediate steps	1.5
Correct analysis of dimensions and number of operations	2.0
Explanation of row-major indexing	1.0
Quality of C code	2.0
JSON discipline	0.5

Table 4. GO-3 rubric and score distribution.

6.4 GO-4 — Latin-square solver, bitmasks, Zig, and a strict output contract

GO-4 is the culmination of the benchmark set. The task requires the model to analyze a small 4x4 Latin-square problem, count empty cells and initial singletons, derive at least four forced steps, explain the bitmask approach, and write a small Zig solver with uniqueness checking.

GO-4 is deliberately the most difficult because it combines four sensitive points: auxiliary numbers, formal logic, a strict syntactic language, and machine-checkable JSON. A model may have the correct final solution and still receive a noticeable point deduction if the initial singletons are counted incorrectly, if Zig is only pseudocode, or if the JSON is inconsistent with the text.

In practice, GO-4 proves to be the strongest filter for models that sound convincing but lack sufficient technical discipline. Therefore, in composite interpretation, it is often considered the most valuable individual test.

GO-4 scoring element	Points / 10
Correct number of empty cells and initial singletons	1.5
Correct candidate sets and bitmasks	1.5
At least four correct forced steps	1.5
Proof / confirmation of uniqueness	1.0
Explanation of the bitmask approach	1.0
Zig code — algorithmic correctness	2.0
Zig code — syntax and structure	1.0
JSON discipline	0.5

Table 5. GO-4 rubric and score distribution.

6.5 GO-5 — seeded Mealy transducer, forensic correction, and Zig 0.16.x+ verifier

GO-5 is designed as an additional integrated test that combines deterministic generation of a formal instance, manual simulation of a formal system, forensic identification of a single error, proof of uniqueness, and modern Zig 0.16.x+ code. Unlike GO-1 to GO-4, where the model receives an already formulated problem instance, here it must first construct the instance from the given seed vector and only then process it manually and algorithmically. This measures a new class of discipline: the ability of the model not only to solve a problem, but also to accurately reproduce its internal generative logic.

The methodological reason for introducing GO-5 is simple. In practice, it turns out that a large number of models can appear very convincing when given a ready-made table, matrix, or input quantities, but lose formal reliability when they must maintain the full chain of consistency: rule generator → system parameters → candidate instances → manual simulation → proof of uniqueness → executable code → machine-checkable JSON. GO-5 is deliberately designed so that one early auxiliary error contaminates all later sections, thereby exposing the model's tendency toward confident continuation on incorrect intermediate results.

The formal domain of GO-5 is a small deterministic Mealy transducer with three candidate input words, one observed output, and the assumption of exactly one symbolic error. This choice is methodologically favorable because the task is small enough for manual verification and dense enough to simultaneously test modular arithmetic, state tracking, Hamming distances, counterfactual checking, and current Zig syntax. Thus GO-5 occupies a position between pure formal logic and systems programming, while adding one new layer: the model must correctly generate the problem it subsequently solves.

GO-5 is also designed as an explicit test against Goodhart-style benchmark degradation. If the public instance becomes known over time, the evaluation signal is preserved not through additional narrative complication, but through audit variants of the same formal generator. In other words, the same benchmark domain can provide multiple instances over different seed values, without changing the rubric and without undermining comparability. The canonical public instance therefore serves as a reference, and the audit variants as protection against mechanical memorization of answers.

The sections of the GO-5 response have strictly divided functions. A-GENERATION measures whether the model accurately follows the rules of system construction. B-MANUAL_SIMULATION measures the discipline of state maintenance across multiple candidates and multiple steps. C-PROOF_OF_UNIQUENESS checks whether the model can distinguish finding one good explanation from actually excluding all alternatives. D-ZIG_CODE checks whether formal reasoning can be converted into an executable, modernly compatible Zig 0.16.x+ program. RESULTS_JSON is the final machine-checkable layer, which must not contradict either the manual or the coding part.

For easier human review and stricter scoring, the manual part of GO-5 is deliberately translated into mandatory mini-tables. The model must present the complete transducer table, the table for generating candidate words, the manual simulation table by candidate, and a summary table of final outputs, paths, and distances. This format reduces the space for the model to obscure an error with narrative and also makes it easier to forensically verify whether candidate_outputs and candidate_distances in the JSON were truly derived from the manual simulation, rather than merely being accidentally suitable for the final conclusion.

GO-5 scoring element	Points / 10
Correct generation of parameters, transducer table, and words	2.0
Correct manual simulation of all three candidates by steps	2.0
Correct summary table of outputs, paths, and Hamming distances	0.5
Proof of uniqueness and inverse check	2.0
Zig code — generation and simulation without hardcoding	1.2
Zig code — Zig 0.16.x+ syntax, I/O, and compatibility	1.0
Zig code — search for all valid explanations and solve Unique logic	0.8
JSON discipline and consistency with B/C sections	0.5

Table 6. GO-5 rubric and score distribution.

7. Horizontal evaluation rules and penalty logic

The benchmark is designed so that the correct final answer alone is not sufficient. Auxiliary numbers, intermediate steps, syntax, and JSON are treated as integral parts of the solution. This prevents a model from guessing the final matrix or number while showing a formally unreliable workflow.

Errors are classified into three classes. Critical errors are those that undermine the essence of the solution: an incorrect minimal number of changes, incorrect manual output, incorrect spike count, incorrect final solution, or a fundamentally incorrect solver. Medium errors include, for example, a correct fraction but incorrect

decimal, a weak proof of minimality, or a syntactic problem that is easily fixable. Small errors are typographical, formatting-related, and stylistic, provided that they do not compromise formal correctness.

A special rule applies to reasoning blocks. If the thinking part is technically separated from the final answer in the user interface, collapsible, and does not contaminate the required A/B/C/JSON output, it is not treated as leakage into the final answer. Penalization is introduced only when reasoning enters the same output block and violates the output contract.

In GO-5, additional classes of errors are singled out that are less frequently seen in the other tests. The first is generator drift: the model makes an error in a transducer parameter or in one component of a generated word, and then consistently solves the wrong problem. The second is simulation drift: one candidate is tracked correctly, but the second or third loses state or output after several steps. The third is a correct-by-coincidence situation, where the model accidentally identifies the winning candidate, but candidate outputs or candidate_distances are incorrect. The fourth is the last-mile error in the Zig 0.16.x+ layer, most often through an outdated pre-0.16 I/O pattern or syntax that appears convincing but is not compatible with the current standard library model.

In the evaluation of GO-5, an additional normative rule applies: section C cannot receive full credit if section B is not correct for all three candidates. Likewise, RESULTS_JSON is considered substantively incorrect if candidate_outputs or candidate_distances are not strictly consistent with the manual simulation, even when the final matching_word_index happens to be correct. This prevents the model from passing solely on the basis of an attractive final conclusion without a valid intermediate chain.

Error class	Typical examples	Effect on the score
Critical error	Incorrect final solution, incorrect minimal number of changes, incorrect key JSON content, materially incorrect solver	Large deduction; the test often falls below 5/10
Medium error	Incorrect auxiliary numbers, insufficiently convincing proof, syntactically weak but recognizable code	Moderate deduction; the test remains passable but not excellent
Small error	Stylistic inconsistencies, minor formatting deviations, minor imprecisions without consequences for the essence	Small deduction

Table 7. Horizontal penalty logic of the benchmark set.

8. Proposal for a standard execution protocol

For the benchmark to be comparable, it is necessary to standardize not only the task texts, but also the execution method. The minimal set of metadata reported with a result includes model name, quantization, prompt template, sampling settings, context, number of generated tokens, TTFT, tokens per second, total time, and, where possible, RAM/VRAM load.

At least one primary run per test is recommended, along with additional reruns if the model shows sensitivity to the template or sampling. All reruns should be preserved, but the representative result per model and test

should be the best quality achieved under stable and documented conditions. Such practice is consistent with experimental work in which one does not report only the first random pass, but the reproducible maximum of the model under a clearly described regime.

For public popularization, it is recommended that the benchmark be published together with canonical prompts, JSON schemas, the scoring scale, and one example of an ideal reference solution for each test. This reduces arbitrariness and strengthens community trust.

- Use the identical task text to compare models of the same generation.
- Record all inference parameters and the prompt/template regime.
- Preserve raw logs and representative final answers.
- For GO-4, report all auxiliary numbers, not only the final matrix.
- Separate the 'all runs' table from the 'best by model' table.
- Clearly indicate on charts whether all runs are displayed or only the best point per model.
- For GO-5, preserve proof of compilation and runtime output for Zig 0.16.x+ code, not only the response text.
- Separate the public canonical GO-5 instance from audit variants with different seed values and different observed outputs, while using the same rubric.
- For GO-5, record whether the model erred in the generator, manual simulation, proof of uniqueness, Zig compatibility, or JSON consistency; this classification is as important as the aggregate score.

9. Proposal for benchmark popularization

For successful popularization, the benchmark must be presented not merely as a set of tasks, but as a clear answer to a real problem in the community: how to distinguish local models that are only rhetorically strong from those that are truly technically reliable. For this reason, promotional materials should emphasize that the GO set measures usability, not only the model's general intelligence.

Second, the benchmark should be promoted through transparency. Publicly available prompts, rubric descriptions, examples of good and poor answers, and a clear result-reporting protocol will make the set credible and easy to adopt. Third, the benchmark should also be presented as an educational tool: through it, users learn why JSON, auxiliary numbers, syntax, and intermediate steps are just as important as final correctness.

For the broader community, it is advisable to use two parallel representations. The first is a technical paper and standardized documentation. The second is a concise promotional "benchmark brief" of several pages with charts, an FAQ section, and examples of typical model errors.

10. Conclusion

GO-1 to GO-5 form a small but methodologically rich benchmark set intended for local and edge LLM/LMM scenarios. Its main strength is that it measures not only the model's intelligence in abstraction, but its technical usability: whether the model follows the format, calculates accurately, explains consistently, provides verifiable JSON, generates an instance from rules, and produces code close to a real tool.

This benchmark should be viewed as a supplement to large public leaderboards, not as a competitor to their role. Its contribution lies in introducing a highly sensitive and transparent assessment of a model's formal discipline — a property of decisive importance in serious local work.

The most important message of this set is: it is not enough for a model to be persuasive; it must be verifiably accurate, structured, and technically disciplined. Precisely for that reason, the GO set has the potential to become a useful reference for the future standardization of local model evaluation.

Annex A. Canonical settings of benchmark tasks

This annex provides standardized, shortened, and promotion-ready formulations of the benchmark tasks themselves, from GO-1 to GO-5. The text may serve as public documentation, as a basis for transferring the benchmark to other systems, and as a reference description for reproduction.

A.1 GO-1 — Canonical formulation

A set of roles {CEO, CFO, OPS, AUD} and a binary permission matrix over the columns {D, A10, A50, VC, VP} are given. The following constraints are specified:

- 1) CEO must have D=1.
- 2) If any role has A50=1, it must also have A10=1.
- 3) Exactly one of the roles {CEO, CFO} must have A50=1.
- 4) OPS must have D=1 and A10=1.
- 5) No role may simultaneously have VC=1 and VP=1.
- 6) Exactly one of the roles {CFO, OPS} must have VC=1.
- 7) CFO must have VP=1.
- 8) AUD may have only D=1, and all other permissions must be 0.

Required:

A-VIOLATIONS: list the violated constraints;

B-CORRECTION: find the minimal number of bit-flip changes and the final matrix;

C-PROOF_OF_MINIMALITY: prove minimality and explain uniqueness;

RESULTS_JSON: deliver machine-checkable JSON with the fields violated_constraints, bit_flips_count, bit_flips, final_matrix, and is_unique_minimum.

A.2 GO-2 — Canonical formulation

A discrete LIF neuron is given:

$$u_t = (3/4) v_t + (1/2) x_t$$

$$s_t = 1 \text{ if } u_t \geq 1, \text{ otherwise } 0$$

$$v_{t+1} = u_t - s_t$$

Initial state: $v_0 = 0$

Input sequence: $x = [1, 1, 1, 0, 1, 1, 0, 1]$

Required:

A-DERIVATION: briefly explain the system dynamics and prove the invariant $0 \leq v_t < 1$;

B-TABLE: provide a table for $t = 0..7$ with x_t, v_t, u_t, s_t , and v_{t+1} in fractions and decimals;

C-CODE: write Python code that computes the same values and prints RESULTS_JSON;

RESULTS_JSON: fields u_fraction, u_decimal, s, v_fraction, v_decimal, spike_count, invariant_holds.

A.3 GO-3 — Canonical formulation

The input matrix X of dimension 4x4 is given:

```
[[1,2,0,1],
 [3,1,2,2],
 [0,1,3,1],
 [2,2,1,0]]
```

The kernel K of dimension 3x3 is given:

```
[[ 1, 0,-1],
 [ 2, 0,-2],
 [ 1, 0,-1]]
```

A valid 2D mathematical convolution with stride=1 and without padding is required.

Required:

A-MANUAL_RESULT: rotate the kernel by 180° and manually compute all output elements;

B-ANALYSIS: explain the output dimensions, the number of multiplications and additions per output element and in total, and row-major indexing;

C-CODE: write the C function `conv2d_valid_3x3` and a small main example;

RESULTS_JSON: `output_h`, `output_w`, `manual_output`, `mul_per_output`, `add_per_output`, `mul_total`, `add_total`.

A.4 GO-4 — Canonical formulation

A 4x4 matrix G with the symbols {1,2,3,4} is given:

```
[[1,0,0,4],
 [0,4,0,0],
 [2,0,4,0],
 [0,3,0,1]]
```

Zero denotes an empty cell. It is specified that every row and every column must contain the symbols 1..4 exactly once.

Required:

A-LOGIC: provide the initial candidate sets after one pass of row and column elimination, count empty cells and singletons, and list at least four forced steps;

B-BITMASKS: explain the encoding of candidates in a u8 mask, singleton recognition, and why counting up to 2 is sufficient for uniqueness checking;

C-ZIG_CODE: write a Zig solver with the functions `popcount4`, `singletonValue`, `candidatesMask`, and `solveUnique`;

RESULTS_JSON: `empty_cells_initial`, `initial_singletons`, `unique`, `solution_count_capped`, `solution`.

A.5 GO-5 — Canonical formulation

A seed vector $V = [11, 7, 19, 4, 23, 8]$ is given, along with a deterministic Mealy transducer with states $\{q_0, q_1, q_2, q_3\}$, input alphabet $\{0, 1, 2\}$, output alphabet $\{A, B, C\}$, and initial state q_0 . System parameters are generated by the rules $a_q = (V[q] + q) \bmod 4$ and $b_q = (V[q+2] + 2q) \bmod 3$, while the next state and output are defined by $\delta(q, x) = (q + a_q + x) \bmod 4$ and $\lambda(q, x) = (b_q + q + 2x) \bmod 3$.

From the same seed vector, three candidate words W_0 , W_1 , and W_2 of length 6 are generated by the rule $W_k[i] = (V[(i + k) \bmod 6] + i + 2k) \bmod 3$. The observed output $Y_{\text{obs}} = \text{"ACCB BB"}$ is also given, under the assumption that it was produced by exactly one candidate and exactly one symbolic error in the output.

Required: A-GENERATION — calculate the parameters, the transducer table, and all three words with intermediate steps; B-MANUAL_SIMULATION — manually simulate all three candidates and calculate state paths, clean outputs, and Hamming distances; C-PROOF_OF_UNIQUENESS — identify the only valid explanation, with an inverse check and exclusion of all alternatives; D-ZIG_CODE — write a Zig 0.16.x+ verifier that generates the instance, simulates all candidates, searches for all valid explanations with exactly one error, and emits RESULTS_JSON; RESULTS_JSON — seed, parameters, words, candidate_outputs, candidate_distances, winning candidate, clean_output, state_path, error tuple, and unique.

Annex B. Recommended result-reporting format

For every publication or internal report, it is recommended that two tables be reported: 'all runs' and 'best by model'. In the first table, all executed runs with full metadata must be preserved. In the second, only the representative best result is shown per model and test. Both tables are necessary: the first for transparency, the second for readability.

The minimal column set for the results table is: Test, Model, Rig, tok/s, Total_s, Quality_10, Note. Where necessary, TTFT, prompt tokens, output tokens, context length, quantization, template regime, flash attention status, and CPU/GPU offload parameters should be added.

Annex C. Proposal for composite interpretation

Test	Recommended weight in the composite	Main signal
GO-1	0.20	Formal logic, minimality, uniqueness
GO-2	0.20	Exact numerics, proof, Python verification
GO-3	0.15	Manual calculation, C implementation, memory model
GO-4	0.20	Integrated test of logic, bitmasks, Zig syntax, and JSON discipline
GO-5	0.25	Deterministic instance generation, simulation discipline, uniqueness, and Zig 0.16.x+ verification

Table 8. Recommended interpretation of the composite benchmark result.

Annex D. Operational guide for scoring GO-5

D.1 Checking the A-section. The evaluator first checks whether parameters a and b were correctly derived from the seed vector and whether W0, W1, and W2 are truly the result of the given formula. If the A-section is wrong, the response can still be read to the end in order to classify the failure, but it cannot be highly scored, because the model is no longer solving the assigned instance but its own incorrect variant of the problem.

D.2 Checking the B-section. Next, it is checked whether all three candidates were manually simulated without skipping steps and whether the state paths, clean outputs, and Hamming distances agree with the tables. If only one candidate is simulated correctly and the second or third is not, this is treated as simulation drift, not as a

minor error. GO-5 is deliberately designed so that all three candidates must be processed, because precisely the third or second candidate often reveals a loss of attention.

D.3 Checking the C-section. The claim of uniqueness is not assessed independently, but only in relation to the correctness of the B-section. The evaluator checks whether the model has not merely found one candidate at distance 1, but has also explicitly excluded all other possibilities. The inverse check is mandatory because it forces the model to move from pure forward reasoning into counterfactual examination of neighboring possibilities.

D.4 Checking the D-section. Zig code is checked on two planes: algorithmic and linguistic. Algorithmically, the code must itself generate the instance, simulate all candidates, and find all valid explanations with exactly one error. Linguistically, the code must be compatible with the Zig 0.16.x+ standard library, without outdated pre-0.16 I/O patterns and without a falsely modern but incompatible API style. Code that understands the problem but does not work out of the box does not belong to the top class of results.

D.5 Checking RESULTS_JSON and the final score. JSON is checked last, but trusted least: it must be the result of the preceding sections, not a decorative summary. If `candidate_outputs` or `candidate_distances` are inconsistent with the manual simulation, the evaluator treats them as a substantive error. The final score is assigned only after the response, code, and JSON are mutually aligned. If the model identifies the final `matching_word_index` but reached it through an incorrect B-section or incompatible Zig, the result is classified as partially correct but operationally unreliable.

The practical recommendation is to maintain, alongside the canonical public GO-5 instance, a small set of audit variants with the same formal rules but different seed vectors and different observed outputs. In this way, the benchmark remains verifiable and standardized, while reducing the risk that models will over time learn only one public example rather than the solving methodology itself.